

Lesson 0001 — From pixels to detections

~45–60 MIN

Foundations

ANCHORED TO: `CUBE_DETECTION_NODE.PY`

Before we study YOLO internals, we need one clear mental model: how a picture becomes a useful answer containing an object name, a location, and a confidence score.

HOW TO USE THIS LESSON

Do not try to memorize every term. Read one section, interact with its visual, then explain the idea in your own words. The goal is understanding the data flow, not reciting a framework tutorial.

PREREQUISITES AND SCOPE

This course assumes you can read code, arrays, functions, and data structures. It assumes no prior machine-learning or computer-vision knowledge. This lesson builds the vocabulary used later; training begins in Lesson 0003, deployment in Lesson 0005, ROS wiring in Lesson 0006, and depth geometry in Lesson 0007.

FROM PIXELS TO DETECTIONS

How one JetRover camera image becomes a structured cube detection

LESSON 0001 • EXPLANATORY OVERVIEW

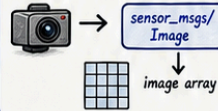
1 Project goal

Turn a camera frame into class + box + confidence for each cube.



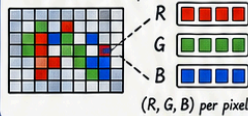
2 ROS image

sensor_msgs/Image enters through /depth_cam/rgb/image_raw. cv_bridge gives the node an image array.



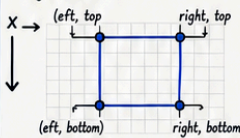
3 Pixels and channels

An image is a grid of numbers. Width x height x channels describes its shape.



4 Coordinates and boxes

The origin is top-left. A box is left, top, right, bottom.



5 Classification vs detection

Classification says what is present. Detection also says where it is.

Classification



Detection



6 One detection

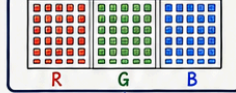
class + bounding box + confidence = one structured prediction.

Detection record	
class:	
box:	(x1, y1, x2, y2)
confidence:	0.92

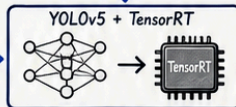
RGB camera frame



pixels / channels

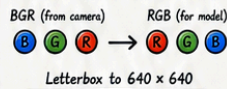


prepare image



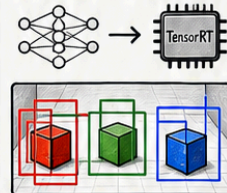
7 Prepare the image

The node handles BGR → RGB and letterboxes to 640 x 640.



8 Model inference

YOLOv5 + TensorRT proposes numerical candidates for the frame.



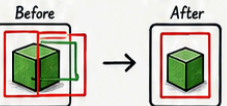
9 Confidence threshold

Keep candidates above the threshold; a score is evidence, not truth.

	0.92	→ KEEP
	0.72	→ KEEP
	0.19	→ REJECT

10 NMS

Remove weaker overlapping boxes so one object has one useful box.



11 Depth and geometry

Depth is a second check. It cannot recover a class the model never proposes.



Publish detections, vendor objects, and an optional debug image.

/cube_detections
/cube_detections/vendor_objects
/cube_detections/debug_image

CODE MAP • ACTUAL DATA FLOW

RGB enters YOLO/TensorRT; depth joins afterward. Pairing is approximate.

SYNC _sync_cb → ApproximateTimeSynchronizer · RGB + depth · slop 0.05 s

RGB _infer: BGR → RGB → _letterbox (resize + pad only) → TensorRT → decode
returns decoded detections; NMS is applied per class

DEPTH geometry_filter runs after inference; invalid or sparse depth → KEEP

OUTPUT _publish_outputs → /cube_detections · /cube_detections/vendor_objects · /cube_detections/debug_image



OBSERVED M5 NOTE

Cubes visible in RGB; at conf=0.50, keep=0 in the tested live scene.



Orientation map for the lesson's main data flow. Use the detailed sections for definitions, limits, and repository-specific evidence. On a phone, click to open and zoom the full-resolution PNG. The landscape artistic variant and the precise portrait technical sheet remain available for comparison.

The question this lesson answers

The JetRover camera continuously produces images. Our project must turn each image into structured information such as:

```
class:      red_cube
location:   a rectangle in the image
confidence: a model score supporting the prediction
```

That result is then published through ROS 2 so that another program—or a human looking at a debug image—can use it.

FOUR WORDS TO KEEP SEPARATE

A **model** is the learned numerical function. **Training** changes its parameters using labelled examples. **Inference** runs the already-trained model on a new image without changing it. **Post-processing** is ordinary code that interprets and filters the model's numeric output. Most confusion later comes from attributing a post-processing decision to “the AI,” or assuming inference keeps learning.

1. What are we trying to recognize?

Our task is to detect three object classes:

- red_cube
- green_cube
- blue_cube

The word **object** matters. The system should not merely say that a color appears somewhere in the picture. It should identify a cube-shaped object and indicate where that object is.

Example camera frame

The boxes and scores below are teaching examples, not a live measurement.



A detection answers: what? where? how strong is the model's evidence?

Figure 1. The result we want: a class label, a bounding box, and a confidence score for each visible cube.

2. What is an image?

A digital image is a rectangular grid of pixels. Each pixel stores numerical color values. A color image has multiple channels—for example, one channel for red, one for green, and one for blue.

In this project, the camera image enters ROS 2 as a `sensor_msgs/Image`. `cv_bridge` converts it into an image array that Python can work with. The current node pairs RGB and depth with `message_filters.ApproximateTimeSynchronizer`; its configured timestamp tolerance is `0.05 s`, so the pair is approximate rather than guaranteed to have identical timestamps. The node asks for a BGR OpenCV image in `cube_detection_node.py:442-445`; before inference, `cube_detection_node.py:508-510` reverses the channel order for the neural network.

For now, the important idea is:

```
camera image = numbers arranged as height × width × channels
```

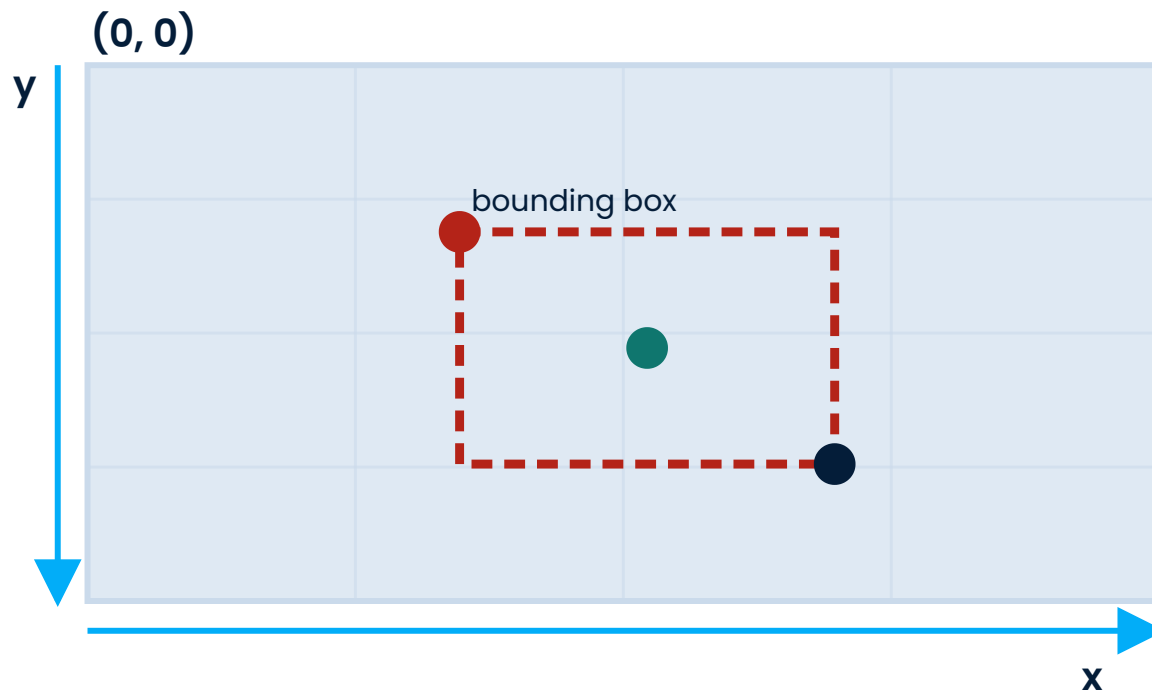
The model does not receive the word “blue.” It receives numerical pixel patterns and has learned that certain patterns often correspond to a blue cube.

Image coordinates

Computer-vision images normally use the top-left corner as the origin:

- x increases as we move to the right;
- y increases as we move down;
- a location is written as a pixel coordinate such as (x, y) .

Interactive: click a point on the image coordinate system



Select a colored point to see its example coordinate.

3. Classification versus object detection

These two terms are related, but they answer different questions.

Classification

"This image contains a red cube."

Detection

"There is a red cube at this location."

Our project

Find multiple cubes and report each location.

Because the JetRover may see several cubes in one frame, we need object detection. A detection includes both a category and a location.

4. The anatomy of one detection

In our project, an internal `CubeDetection` contains:

<code>class_name</code>	→ which class was predicted
<code>score</code>	→ the model's confidence score
<code>box</code>	→ the pixel rectangle around the object
<code>reason</code>	→ why the later geometry step kept it

The first three fields are the basic object-detection idea. The `reason` field is specific to this project's later depth/geometry filtering.

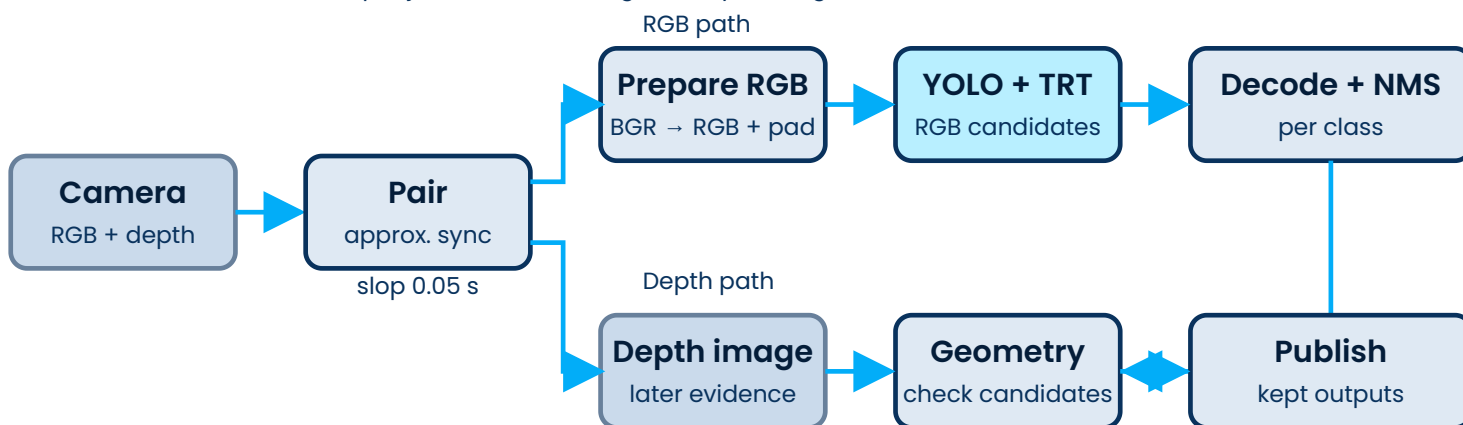
CONFIDENCE IS EVIDENCE, NOT TRUTH

A score of `0.80` does not guarantee that the prediction is correct. It means the model has produced relatively strong evidence according to its learned scoring system. A model can be confidently wrong, especially when the new camera scene differs from its training images.

5. The complete runtime pipeline

Here is the full process for one approximately synchronized RGB/depth pair selected with a configured slop of `0.05 s`:

Approximate pairing: `ApproximateTimeSynchronizer · slop 0.05 s`.
RGB enters YOLO/TensorRT; depth joins afterward at geometry filtering.



Wrong whole-frame depth drops the frame; sparse per-box depth keeps a candidate with a quality reason.

Figure 2. The implementation-level mental model: approximate RGB/depth pairing, an RGB inference branch, and a later depth/geometry check.

Interactive: inspect one pipeline stage

Camera: The vendor camera publishes RGB and depth images. The node receives an approximately synchronized pair through `ApproximateTimeSynchronizer` with a configured `0.05 s` slop.

6. What happens after the model predicts?

The neural network usually produces more candidate boxes than we want to publish. Some candidates are weak. Several candidates may describe the same physical cube with slightly different rectangles.

Two common ideas help:

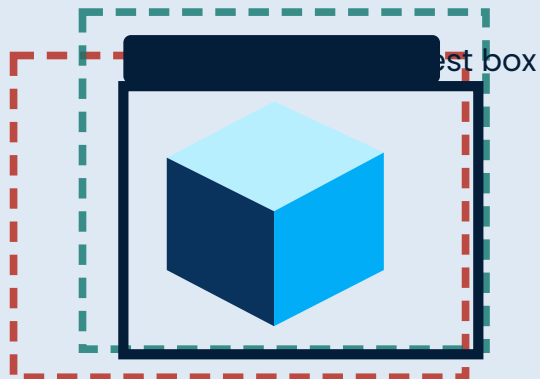
1. **Confidence thresholding:** discard candidates below the selected score threshold.
2. **Non-maximum suppression (NMS):** within each class, when several boxes overlap the same object, keep the strongest useful box and remove weaker duplicates. The current implementation applies NMS per class, so overlapping boxes from different classes are not suppressed together.

Today we only need the intuition. Lesson 2 will explain how the current YOLO output is decoded and how NMS is implemented in `_decode_yolov5_output()`. In this node, `_infer()` returns decoded detections after confidence filtering and per-class NMS; it does not pass the raw engine tensor directly to `_sync_cb()`.

Interactive: overlapping candidates before and after NMS

Before NMS: several guesses

All three boxes point at the same physical cube.



The model can make multiple overlapping guesses. NMS reduces same-class duplicates to a cleaner result; different classes are handled separately.

Confidence thresholding

The confidence threshold is a decision boundary. Raising it usually makes the system more selective; lowering it usually allows more candidates through. Neither direction automatically makes the model correct.

Interactive: change the confidence threshold

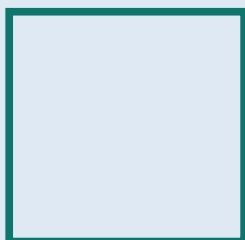
Threshold **0.50**

Illustrative candidates

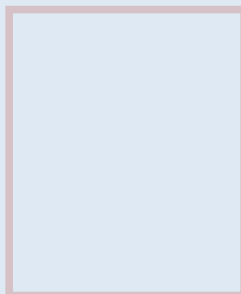
0.82



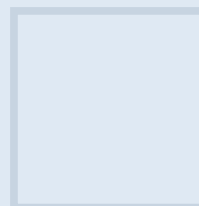
0.58



0.41



0.19



Candidates below the threshold are faded.

At threshold 0.50, 2 of 4 illustrative candidates remain visible.

7. Map the mental model to the real code

The current node has one main callback for an approximately synchronized RGB/depth pair and several focused helpers:

Code

`_letterbox()`

lines 72–94

`_decode_yolov5_output()`

lines 97–145

`_sync_cb()`

lines 437–502

`_infer()`

lines 504–530

`_publish_outputs()`

lines 532–541

Plain-language job

Resize and pad the already RGB frame while remembering how to map boxes back to the original image. It does not perform the BGR → RGB conversion.

Decode model numbers into image boxes, class names, and scores; apply the confidence threshold and per-class NMS.

Receive an approximately synchronized RGB/depth pair via `ApproximateTimeSynchronizer` (slop 0.05 s), then coordinate inference, geometry filtering, publishing, and timing.

Convert BGR → RGB, call `_letterbox()`, normalize/transpose, execute TensorRT, and return decoded detections—not raw engine output.

Publish kept detections to `/cube_detections` and `/cube_detections/vendor_objects`; optionally publish the debug image to `/cube_detections/debug_image`.

THE MODEL IS NOT THE WHOLE APPLICATION

YOLO/TensorRT performs one important stage on the RGB branch: producing predictions. The ROS 2 node receives an approximately synchronized RGB/depth pair, prepares RGB for the engine, interprets the decoded output, applies the later depth/geometry check, and publishes messages. The boundary matters: a wrong-shape or wrong-type whole depth frame makes the callback return before inference, while missing or sparse statistics for one otherwise valid candidate box take the geometry helper's fail-open branch.

Depth is a later evidence check, not a replacement detector. With valid, sufficiently sampled depth, geometry can reject a candidate that does not look like a raised cube-like object. If the geometry statistics are invalid or fewer than the configured minimum number of valid depth pixels are available, this implementation keeps the candidate with a low-depth-quality reason. A whole depth frame with the wrong shape or type is skipped before publishing.

8. A real example from this project

The following two images come from the M5 live test. A human can see three small cubes in the RGB frame. The debug overlay says `keep=0` because no candidate passed the current model and geometry pipeline at the selected threshold. [M5 evidence]



RGB input: the three cubes are visibly present.



Debug output: `conf>=0.50` , filter on, `keep=0` .

This is not a contradiction. It is a useful engineering observation: the image, the model's candidate predictions, the confidence threshold, and the geometry filter are separate stages. A later lesson will explain why the current model performs better on its original validation images than on this real JetRover view.

9. Check your understanding

Which sentence describes object detection?

1. ☐ "A red cube is somewhere inside this image."
2. ☐ "A red cube lies inside this predicted rectangle."
3. ☐ "This image contains red, green, and blue channels."
4. ☐ "The camera captured this image at thirty hertz."

What does a confidence score mean?

1. ☐ It is the measured probability that the object physically exists.
2. ☐ It is the estimated distance between camera and detected object.
3. ☐ It measures how strongly the model supports this particular prediction.
4. ☐ It counts how many pixels fall inside the predicted box.

Why do we need NMS?

1. ☐ To suppress weaker overlapping boxes for the same predicted class.
2. ☐ To merge overlapping boxes from every class into one label.
3. ☐ To convert model-space boxes back into original-image coordinates.
4. ☐ To remove every candidate whose score is below threshold.

10. Tiny read-only exercise

No code change is required for this exercise.

1. Look at Figure 1 and describe one detection using the three words **class**, **box**, and **confidence**.
2. Open `recognition_of_different_colored_cubes/cube_detection_node.py` and find `_letterbox()`, `_infer()`, and `_publish_outputs()`. Write one sentence describing the job of each.
3. Look at the two M5 images above. Explain why "three cubes are visible" and "keep=0" can both be true.

11. Explain it back

Try to explain the following without reading the previous sections:

“The camera produces RGB and depth images. The node pairs them approximately, prepares RGB, and gives only RGB to the detector. The detector proposes objects, and each proposal has a class, a box, and a score. Weak same-class duplicates are filtered, then depth checks the candidates when usable. A malformed whole depth frame drops that frame, while sparse per-box evidence keeps that candidate under the current fail-open rule. The remaining information is published as ROS 2 messages and, optionally, a debug image.”

If you can explain that flow, you understand the central idea of this lesson. The details of the neural-network tensor come next.

12. Project note and next lesson

The current model and node use the class order `0: blue_cube , 1: green_cube , 2: red_cube`. The project-definition document still shows a different class-ID table, so that documentation conflict must be reconciled before another fine-tuning run. We will not silently guess when class IDs matter.

Where the postponed details live: the candidate tensor and decoder belong to Lesson 0002; learning and validation to Lesson 0003; export to Lesson 0005; ROS integration to Lesson 0006; depth geometry to Lesson 0007; and evaluation to Lesson 0008. CNN derivations remain outside this applied nine-lesson track because they are not required to trace or improve the present system.

Lesson 2 will answer:

How does YOLO turn one image into many candidate predictions, and how does the current code decode those predictions into final boxes?

Primary references

- Ultralytics YOLOv5 documentation — the model family used by this project.
 - PyTorch beginner workflow — data, models, optimization, and saved models.
 - ROS 2 topics — the publish/subscribe model used for camera and detection streams.
 - Current detection node source — the implementation this lesson maps to.
 - Learning plan — the complete teaching sequence.
-

CONTINUE WITH THE TEACHING AGENT

Ask the agent to clarify anything uncertain, challenge your explanation with a counterexample, or assess your explain-back. A learning record should be created only after you demonstrate the idea in your own words.

Course home

Quick reference

Mission

Next lesson →